UTA SUMO ROBOTICS CUP 2026

GUÍA COMPLETA DEL SISTEMA

Manual funcional, técnico y de operación

FRONTEND + BACKEND + BASE DE DATOS

Qué hace la aplicación, cómo se conectan sus módulos y qué necesita una instalación nueva para operar. Esta guía describe el código vigente del repositorio y señala las dependencias de configuración externa.

Ficha	Estado documentado
Proyecto	UTA SUMO ROBOTICS CUP 2026
Frontend	React + Vite · interfaz pública, participante y personal autorizado
Backend	FastAPI + SQLAlchemy · API REST, WebSockets y servicios de archivos
Base de datos	Microsoft SQL Server · migraciones ordenadas y trazables
Rama revisada	feature/backend-core
Revisión de fuentes	26 de octubre de 2026

Alcance y precisión

La guía distingue entre código implementado y servicios que dependen de credenciales externas. Google requiere un Client ID válido y orígenes OAuth autorizados; el correo real requiere SMTP configurado. En local, las claves se guardan en archivos .env ignorados por Git. Nunca se incluyen secretos ni contraseñas en este documento.

Contenido

Parte	Contenido
1	Resumen y arquitectura
2	Perfiles de usuario y experiencia
3	Registro, acceso y seguridad
4	Flujo de inscripción
5	Tarifas, pagos y comprobantes
6	Administración y catálogos
7	Competencia, marcador y cronómetro en vivo
8	Reportes, búsquedas y paginación
9	Reglamentos y archivos
10	Modelo de datos y transacciones
11	Migraciones e instalación en otra máquina
12	Pruebas y verificación manual
13	Cambios incorporados en backend y frontend
14	Pendientes operativos y recomendaciones de despliegue

Atajo para probar

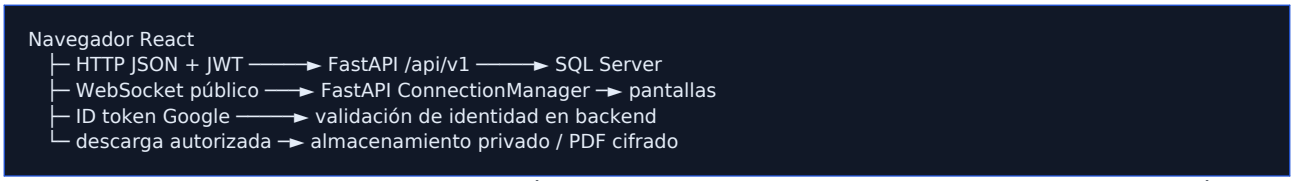
Backend: http://127.0.0.1:8000 · Swagger: /docs · Salud: /api/v1/health. Frontend: Vite en http://localhost:5173. El flujo de participante comienza con acceso Google; el personal usa acceso administrativo con email y contraseña.

1. Resumen y arquitectura

El producto permite que los participantes creen equipos y robots, presenten inscripciones y comprobantes; que el personal administre el evento y revise pagos; y que jueces actualicen la competencia para que los resultados se reflejen en pantallas públicas. El frontend consume el backend por HTTP y recibe actualizaciones en tiempo real por WebSocket.

Capa	Tecnología/función
Interfaz web	React, React Router y Vite. Formularios, paneles, resultados y navegación responsive.
API	FastAPI, Pydantic, SQLAlchemy. Contratos JSON, autenticación, validación y permisos.
Persistencia	SQL Server. Integridad referencial, restricciones, índices, auditoría y transacciones.
Archivos privados	Comprobantes cifrados en almacenamiento privado; reglamentos PDF cifrados dentro de SQL Server.
Tiempo real	WebSocket para combate actual y clasificación en vivo.
Servicios externos	Google Identity Services para identidad de participantes; SMTP para correo transaccional.

Un recorrido simplificado



El frontend no es la autoridad para reglas críticas. El backend vuelve a validar permisos, propiedad, límites del roster, tarifas, estados, tamaño y contenido de archivos; SQL Server añade llaves, checks, unicidad e índices.

2. Perfiles de usuario y experiencia

Perfil	Qué ve y qué puede hacer
Visitante	Inicio, combate público, información de participantes, inscripción y alta/acceso de participante.
PARTICIPANT	Inicio del panel, combate en vivo, inscripción de equipo; consulta sus equipos, órdenes, comprobantes y datos propios.
EVENT_MANAGER	Herramientas operativas, inscripciones, formularios y competencia; administra rondas, enfrentamientos, estados y puntuaciones. Sin descarga/revisión financiera.
ADMIN	Administración completa: catálogos y usuarios, revisión de pagos/comprobantes, reportes financieros, reglamentos y operaciones.

Navegación y vistas

- Para visitantes, el menú superior ofrece Inicio, Ver combate, Participantes, Inscripción, Crear cuenta y Personal autorizado.
- Para participante autenticado, Inicio aparece primero; se ofrecen Combate en vivo, Inscribir equipo y Cerrar sesión. El panel agrupa Inicio, órdenes, categorías, reglamento y equipo.
- Para administrador autenticado, la navegación superior se reduce al Panel de administración; las secciones del panel quedan en el menú lateral. En pantallas pequeñas el menú se adapta.
- Los enlaces usan iconos funcionales y etiquetas claras. El nombre del equipo es el identificador principal visible en la tabla administrativa; el código técnico queda como referencia secundaria.

Los permisos también se verifican en API. Ocultar un botón en React no sustituye autorización del servidor. Los roles activos son ADMIN, EVENT_MANAGER y PARTICIPANT; las cuentas privilegiadas se aprovisionan por operador, no desde registro público.

3. Registro, acceso y seguridad

Participantes: la interfaz vigente presenta acceso con Google. La primera autenticación verificada crea la cuenta de participante y su perfil asociado; los siguientes accesos recuperan la identidad existente. Se pide una institución para crear el perfil. El backend no recibe la contraseña Google: verifica un ID token, su firma, emisor, expiración, audiencia y correo verificado. La identidad externa se vincula por el identificador estable de Google (sub).

Personal: entra por la vista administrativa con email y contraseña local. Un operador crea cuentas privilegiadas mediante el script interactivo backend/uta_sumo_api/scripts/provision_operator_user.py. El registro público no puede asignar roles privilegiados.

Control	Comportamiento actual
JWT	Bearer token firmado HS256; expiración configurable, 30 minutos por defecto. Sin refresh token. Se guarda en sessionStorage del navegador.
Contraseña local	Argon2id; mínimo de 15 caracteres en la política documentada. Bcrypt heredado solo para validar hashes existentes y migrarlos al autenticar.
Google	Client ID frontend y backend deben coincidir. Sin ID token válido no se crea sesión. Google password no se recopila.
Correo	Formato y DNS/dominio pueden validarse, pero no equivalen a verificar control del buzón.
Cuentas existentes	No se vincula una cuenta local silenciosamente solo porque coincida el correo; se usa el flujo autenticado de vinculación.
Claves de aplicación	SECRET_KEY firma JWT. PII_ENCRYPTION_KEY protege datos; conservar ambas fuera de Git y respaldarlas de forma segura.

Configuración Google local

```
Backend: backend/uta_sumo_api/.env
  GOOGLE_CLIENT_ID=<OAuth Client ID tipo Web>

Frontend: frontend/uta-sumo-web/.env
  VITE_GOOGLE_CLIENT_ID=<EL MISMO CLIENT ID>
  VITE_API_BASE_URL=http://127.0.0.1:8000

Google Cloud Console: Authorized JavaScript origin = http://localhost:5173
Reiniciar FastAPI y Vite después de cambiar variables.
```

El Client ID es público y se configura en ambos lados. No se necesita Client Secret en este flujo de ID token. Para el despliegue hay que registrar el dominio publicado y usar HTTPS.

4. Flujo de inscripción

El responsable debe iniciar sesión. El backend identifica la cuenta propietaria desde el JWT; el cliente no puede enviar otro user_id para tomar propiedad ajena. El asistente del frontend reúne evento y categoría, equipo/robot, roster y confirmación. Las reglas de participantes y coaches vienen de la relación evento-categoría, no de una constante visual.

Paso	Frontend	Backend / SQL
1 · Acceso	Continuar con Google e institución para primer perfil.	Verifica token Google y crea/recupera la cuenta PARTICIPANT.

Paso	Frontend	Backend / SQL
2 · Preparar	Carga eventos, categorías, tarifas y reglas; permite elegir recursos propios.	API filtra recursos por propietario y entrega límites vigentes.
3 · Roster	Solicita nombres, apellidos, institución y celular; valida datos y duplicidad localmente.	Valida rangos/roles, normaliza datos y cifra PII antes de guardar.
4 · Confirmar	Muestra resumen y tarifa calculada por la API.	Una transacción crea inscripción, roster, pago PENDING, historial y documento de solicitud.
5 · Continuar	Enseña código/nombre del equipo, estado y orden de pago.	Responde identificadores, estado del pago y URL autorizada del documento.

Regla vigente para la categoría oficial

Las migraciones 23 y 25 declaran valores diferentes: la migración posterior 25 prevalece y deja Mini Sumo Amateur RC de UTA SUMO ROBOTICS CUP 2026 con exactamente dos participantes y un coach. El frontend construye el roster usando la regla recibida; el backend la valida nuevamente. Datos históricos no se reescriben por el cambio de regla.

Duplicidad y pertenencia

- Nombres de equipos y robots se normalizan para comparar, tienen índices/constraints de unicidad y se validan antes de persistir.
- El robot no puede volver a inscribirse en la misma categoría y evento; el equipo puede tener varios robots y competir en diferentes categorías/eventos según la regla del sistema.
- Una inscripción debe enlazar evento, categoría, equipo y robot válidos y propiedad autorizada.

Los límites y relaciones se validan tanto en la interfaz como en la API. La garantía final ante solicitudes concurrentes se apoya en SQL Server, transacción y constraints.

5. Tarifas, pagos y comprobantes

El precio de la inscripción se consulta/calcula con la tarifa vigente y se guarda como snapshot en el pago para conservar el monto cobrado aunque la tarifa cambie después. Cada inscripción tiene un solo pago; puede haber varios intentos de comprobante tras rechazo y se conserva su historial.

Estado / acción	Efecto esperado
PENDING	Orden creada; participante aún no ha presentado evidencia.
Subir comprobante	PARTICIPANT carga PDF, PNG o JPEG. Archivo privado, cifrado AES-GCM y asociado al pago.
PAYMENT_REVIEW	Comprobante disponible para revisión autorizada; no se confirma automáticamente que el dinero llegó.
PAID	ADMIN verifica el pago; la inscripción puede avanzar según su flujo operativo.
REJECTED	Se registra motivo; vuelve a PAYMENT_PENDING para que el participante corrija y reenvíe.
Descarga	Se comprueba la propiedad o el rol de staff autorizado antes de descifrar/servir.

Límites y controles

- Tipos aceptados: PDF, PNG y JPEG; compara extensión, MIME y firma del contenido. El nombre físico usa identificador aleatorio.
- Límite por defecto: 5 MiB; configuración MAX_RECEIPT_BYTES con máximo documentado de 20 MiB.
- No publicar el directorio de comprobantes como estático. En producción montar almacenamiento privado y respaldarlo junto con sus metadatos.

• El botón “Ver comprobante” abre una vista previa/descarga con sesión y permiso; la decisión administrativa mantiene auditoría.

Correo y solicitud de pago

La creación genera un PDF de solicitud de pago y una fila en EmailOutbox dentro de la transacción. El documento aclara que no es factura fiscal ni confirma pago. SMTP se intenta después del commit; si no hay credenciales, no se finge envío y la entrega queda pendiente/fallida con posibilidad de reintento por personal. Para correo real se configura SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD, SMTP_FROM_EMAIL y SMTP_STARTTLS en backend .env.

Política importante: SMTP PASSWORD es secreto; no debe copiarse en frontend, PDF público, repositorio ni mensaje de commit.

6. Administración y catálogos

El panel organiza inscripción, marcador en vivo, formularios y personal autorizado. La administración puede buscar/filtrar registros y revisar información operativa; la gestión de categorías, eventos, instituciones, tarifas y roles se expone por endpoints administrativos. Cada endpoint aplica rol y valida referencias.

Módulo	Responsabilidad funcional
Inscripciones	Vista paginada, filtros, estados, tarifa, equipo/robot, roster y estado de pago; exportación CSV. ADMIN revisa comprobante y decisión.
Marcador en vivo	Elegir/gestionar encuentro, lados, robots, estado, detalles, puntuaciones y cronómetro.
Formularios / reglas	Configuración administrativa de campos/reglas de registro y límites por evento/categoría según API habilitada.
Personal autorizado	Alta de cuentas privilegiadas y roles bajo control de operador/ADMIN según endpoint. El frontend no eleva el rol de un participante.
Catálogos	Eventos, categorías, instituciones, tarifas y estado activo; cambios sujetos a autorización.
Reglamentos	Carga/edición de borrador, publicación, archivo e historial de versiones.

Estados de inscripción

El código mantiene estados de registro con transiciones auditables. De forma general, el flujo de pago pendiente pasa a revisión de pago; al verificarse PAID, el operador puede aprobar, observar o rechazar la inscripción. Si se rechaza el pago, retorna a pago pendiente. Consulta las rutas de revisión y el historial para el contrato exacto vigente; no se debe editar directamente SQL para saltar transiciones.

7. Competencia, marcador y cronómetro en vivo

El sistema mantiene ciclos del evento, rondas y enfrentamientos en SQL Server. Los movimientos de puntaje se guardan como entradas append-only para conservar auditoría; el total no puede ser negativo y los reintentos usan Idempotency-Key. Solo se puntúa en competencia LIVE y con inscripción aprobada. Empates pueden compartir posición.

Canal	Ruta	Uso
Lectura pública	GET /api/v1/competition/combate-actual	Carga inicial para la pantalla pública; nombres, instituciones, puntos y estado derivan del dato persistido.
Escritura protegida	PUT /api/v1/competition/combate-actual	ADMIN/EVENT_MANAGER actualizan encuentro/estado/marcador; backend persiste y publica luego del commit.

Canal	Ruta	Uso
Cronómetro	PUT /api/v1/competition/combate-actual/cronometro	Configura duración, dirección UP/DOWN y habilitación.
Control de reloj	POST /api/v1/competition/combate-actual/cronometro/{start|pause|reset}	Iniciar/reanudar, pausar o reiniciar con permisos de staff.
WebSocket del combate	ws://127.0.0.1:8000/ws/combate-actual	Instantánea inicial y broadcast público tras los cambios persistidos; desconexiones se retiran.
WebSocket de tabla	ws://127.0.0.1:8000/api/v1/events/{event_id}/live	Estado y clasificación, sin PII ni pagos; publica instantánea y detecta cambios con sondeo.

Cómo se sincroniza el reloj

El servidor conserva duración, dirección, estado y marca temporal UTC. La pantalla calcula/representa tiempo usando la instantánea recibida; no controla el reloj oficial desde su reloj local. La cuenta regresiva se detiene en cero; la cuenta progresiva termina al alcanzar la duración configurada. Pausa, reinicio y cambios de estado se persisten. Para la competición activa se habilita el reloj según la configuración vigente.

Límite de despliegue

El ConnectionManager del combate vive en el proceso FastAPI. Un único worker transmite correctamente entre los clientes conectados a esa instancia. En varios workers o servidores se debe agregar broker compartido (por ejemplo Redis/pub-sub) para fan-out entre procesos; sin broker, una conexión no recibe broadcasts emitidos desde otra instancia.

8. Reportes, búsquedas y paginación

Los reportes agregados se calculan desde SQL; las tablas grandes emplean paginación en el servidor y un límite máximo de página. Los listados de reportes protegen información de contacto y referencias financieras.

Ruta de reportes	Permiso y contenido
GET /api/v1/reports/overview	ADMIN / EVENT_MANAGER: KPIs agregados por evento. Datos de usuarios/finanzas completos solo para ADMIN.
GET /api/v1/reports/registrations/by-status	ADMIN / EVENT_MANAGER: conteos por estado.
GET /api/v1/reports/registrations/by-category	ADMIN / EVENT_MANAGER: distribución por categoría.
GET /api/v1/reports/registrations/trend?from_date=…&to;_date=…	ADMIN / EVENT_MANAGER: tendencia en rango de fechas ISO.
GET /api/v1/reports/payments/summary	ADMIN: conteos e importes agrupados por moneda y estado.
GET /api/v1/reports/registrations/details?page=1&limit;=50	ADMIN / EVENT_MANAGER: filas operativas paginadas, sin datos de contacto.
GET /api/v1/reports/payments/details?page=1&limit;=50	ADMIN: filas financieras paginadas, sin referencia bancaria ni PII.

Paginación práctica

Parámetros de listado: page (desde 1) y limit (hasta 100). El contrato incorpora items, page, page_size, total y total_pages; algunos endpoints también añaden encabezados X-Total-Count, X-Page, X-Page-Size y X-Total-Pages. El frontend debe renderizar controles y conservar filtros al cambiar de página. Ordenar y filtrar desde SQL reduce memoria, ancho de banda y exposición de filas no necesarias.

Panel profesional sugerido

- KPIs por evento: inscritas, aprobadas, pendientes, pagos confirmados y monto por moneda.
- Gráficos: estado de inscripción, distribución por categoría y tendencia diaria/semanal.
- Tabla detalle con búsqueda, filtros de evento/categoría/estado/fecha, paginación y exportación explícita.
- Mostrar hora y zona horaria; no sumar monedas distintas sin conversión; no exponer email, celular ni identificador bancario en reportes operativos.

La API entrega datos/reportes y controles de permisos; cualquier gráfico o descarga adicional debe consumir ese contrato, no recalcular desde datos parciales del navegador.

9. Reglamentos y archivos

Los PDFs de reglamento se almacenan cifrados en SQL Server, no como recurso estático del frontend. Se mantienen evento, categoría opcional, título, versión, vigencia, estado, hash SHA-256 y auditoría. El usuario público ve solo documento publicado y vigente; el administrador puede cargar borrador, publicar versión o archivar conservando historial.

Operación	Endpoint
Listar vigentes por categoría	GET /api/v1/events/{event_id}/regulations?category_id={id}
Ver PDF publicado	GET /api/v1/regulations/{regulation_id}/file
Listar versiones (ADMIN)	GET /api/v1/admin/events/{event_id}/regulations
Cargar borrador PDF (ADMIN)	POST /api/v1/admin/events/{event_id}/regulations (multipart/form-data)
Editar metadatos de borrador	PATCH /api/v1/admin/regulations/{id}
Publicar o archivar (ADMIN)	POST /api/v1/admin/regulations/{id}/publish · /archive
Historial	GET /api/v1/admin/regulations/{id}/history

Validación PDF

El tamaño máximo por defecto es 20 MiB (MAX_REGULATION_BYTES); valida estructura, entre 1 y 500 páginas, documento sin contraseña y restricciones contra acciones automáticas/adjuntos incrustados. La migración 31 realiza backfill del contenido histórico hacia FileContent cifrado; migración 32 finaliza el esquema de almacenamiento en base. Conserva PII_ENCRYPTION_KEY: perderla impediría descifrar contenido anterior.

10. Modelo de datos y transacciones

Grupo de tablas / entidad	Propósito y relación principal
Users, Roles, Participants, UserExternalIdentities	Credenciales, rol, perfil participante e identidad OIDC (Google sub).
Institutions, Events, Categories, EventCategories, Tariffs	Catálogos, eventos, vínculo categoría-evento, reglas de cupos y tarifa vigente.
Teams, Robots	Recursos con institución, identificadores normalizados y propietario.
Registrations, RegistrationParticipants	Inscripción por evento/categoría/equipo/robot y roster con rol/posición.
Payments, PaymentReceipts, EmailOutbox	Snapshot de tarifa, estado del pago, metadata de comprobantes y cola persistente de correos.
StatusHistory, UserSecurityAudit	Historial de cambios de inscripción y auditoría de acciones sensibles.
CompetitionRounds, CompetitionMatches, ScoreEntries	Estructura de torneo, partidos, configuración del reloj y libro append-only de puntos.
ActiveCombat, ActiveCombatHistory	Encuentro seleccionado en la pantalla pública y cambios auditados.
EventRegulations, RegulationHistory	Metadatos/archivo PDF cifrado, versiones y trazabilidad.
SchemaMigrations	Scripts aplicados y hash para controlar versión e integridad.

Atomicidad y consistencia

- Crear inscripción: roster, registro, pago, historial y orden PDF/outbox se coordinan en transacción; cualquier fallo SQL revierte el bloque transaccional.
- Subir comprobante: se comprueba rol/propiedad, tipo, tamaño y contenido antes de persistencia; descarga exige autorización de nuevo.
- Cambio de puntaje/marcador: SQL confirma el cambio; el broadcast WebSocket ocurre después del commit para no anunciar datos no persistidos.
- Publicar reglamento: la versión anterior de ese mismo alcance se archiva dentro de operación atómica y se mantiene auditoría.
- Índices y constraints protegen FK, estados, duplicidad y valores de dominio; los endpoints regresan errores sanitizados ante fallos de base.

Cifrado: la aplicación cifra campos personales y archivos sensibles a nivel de aplicación. Esto complementa, pero no reemplaza, cifrado de disco/backup/TDE de SQL. El backup operativo debe contener datos y claves protegidas con acceso separado.

11. Migraciones e instalación en otra máquina

El repositorio usa los SQL de database/scripts y database/seeds. El ejecutor scripts/migrate.py los ordena, separa lotes GO, registra hashes y aplica cada migración de esquema/datos en una transacción cuando SQL Server lo permite. CREATE DATABASE se gestiona aparte porque SQL Server no lo admite dentro de esa transacción. El uso normal es aditivo y preserva datos; reset es explícito y solo local.

Etapa	Migraciones relevantes
Modelo base	01 crear base; 02 tablas; seed 02 datos iniciales; 03 relaciones; 04 índices; 05 restricciones.
Seguridad/identidad	06–09 cifrado de identidad, restricciones, índices de paginación e identidades Google.
Competencia	10–12 scoring, historial de estado y normalización final de roles.

Etapa	Migraciones relevantes
Inscripción/correo	13–17 posiciones de roster, delivery/outbox, capacidad cifrada, versión de documento y filtros.
Evento y marcador	18–24 reglamentos, combate actual, selección de robots, evento 2026, reglas categoría y detalles públicos.
Reglas y reloj	25 roster/reloj, 26 reloj habilitado, 27 dirección UP/DOWN, 28 reloj activo en match LIVE.
Duplicidad y archivos	29–30 lookup normalizado/unicidad de nombres de recursos; 31–32 reglamento PDF en SQL cifrado y cierre del backfill.

Configuración local Windows

```
1) Arrancar SQL Server Express y confirmar ODBC Driver 18.
2) En backend/uta_sumo_api: crear/usar .venv e instalar requirements.txt.
3) Copiar .env.example de la raíz a backend/uta_sumo_api/.env.
4) Definir SECRET_KEY y PII_ENCRYPTION_KEY; revisar conexión SQL y CORS.
5) Ejecutar .venv\Scripts\python.exe scripts\migrate.py
6) .venv\Scripts\python.exe -m uvicorn main:app --reload
7) En otra terminal: frontend/uta-sumo-web; npm install; npm run dev
```

Claves: crea SECRET_KEY aleatoria ≥32 caracteres. PII_ENCRYPTION_KEY debe codificar 32 bytes aleatorios en base64url, según la implementación/configuración vigente. No reemplazar las claves de una instalación con datos cifrados; usar política segura de rotación/re-cifrado. El .env nunca se sube a Git.

Cuentas de prueba

La guía de backend indica que la migración crea perfiles demo locales por rol; las contraseñas generadas por bootstrap se muestran solo al operador en el proceso local y no se recuperan desde hash. Para aprovisionar un usuario privilegiado propio, usar provision_operator_user.py. No documentar contraseñas compartidas y desactivar el bootstrap de demo en producción.

12. Pruebas y verificación manual

El repositorio contiene pruebas automatizadas para contratos y reglas. En el checkout deben ejecutarse en el entorno del backend; las pruebas que usan SQL Server requieren base/migraciones locales. No se debe usar la base de datos de producción para pruebas destructivas.

Arranque y salud

```
GET http://127.0.0.1:8000/api/v1/health
GET http://127.0.0.1:8000/health/live
Abrir http://127.0.0.1:8000/docs
```

Flujo de verificación recomendado

N°	Prueba	Resultado a comprobar
1	Abrir inicio y combate público	Sin sesión funciona lectura pública; la UI muestra combate actual o estado vacío válido.
2	Participante: continuar con Google	OAuth abre el dominio/origen permitido; primer acceso crea cuenta; segundo autentica. No aparece contraseña Google en solicitudes a la app.
3	Crear equipo y robot	Nombre repetido debe ser rechazado incluso con diferencia de espacios/capitalización; propietario es la cuenta.
4	Enviar inscripción	Roster corresponde a reglas y categoría; precio coincide; se crea estado de pago pendiente y solicitud PDF.
5	Subir comprobante	Archivo permitido y debajo del límite pasa; tipo falso/exceso de tamaño se rechaza; propietario no puede acceder a inscripción ajena.
6	Revisión ADMIN	Ver comprobante requiere token ADMIN; decisión cambia estado y deja historial. EVENT_MANAGER no recibe acceso financiero.
7	Puntuar/cambiar reloj	Solo personal autorizado puede escribir; pantalla pública recibe broadcast después de commit; timer persiste y dirección opera.
8	Reportes	Page/limit respeta máximo; totales/páginas correctos; permisos restringen contenido financiero.
9	Reglamento	ADMIN carga y publica; visitante descarga solo versión vigente publicada.

Postman

backend/uta_sumo_api/postman guarda colección/entorno para ejecutar solicitudes HTTP. Importa ambos, configura base_url, crea token mediante la ruta de autenticación correspondiente y usa Bearer token en operaciones protegidas. Para Google no inventes ID token: inicia sesión con Google real en frontend o usa token emitido para el mismo Client ID. WebSocket se prueba con cliente WS de Postman o navegador compatible; una prueba HTTP no valida el canal.

No se describe aquí un resultado de ejecución de pruebas que no se haya corrido para esta documentación. Los pasos son el protocolo reproducible para verificar el checkout actual.

13. Cambios incorporados en backend y frontend

La rama revisada contiene una evolución integrada del sistema. Esta lista agrupa las funciones que se observan en el código y migraciones actuales; no implica que los servicios externos estén configurados en cualquier instalación nueva.

Backend

- Autenticación por JWT, Google Identity token para participante, roles activos, permisos por propietario y aprovisionamiento de personal autorizado.
- Cifrado de información personal/correos y cifrado de comprobantes/reglamentos; HMAC/índices para búsqueda de algunos identificadores sensibles.
- Modelo de evento 2026 y reglas de inscripción/roster, asociación de propiedad, control de duplicidad de equipos y robots.
- Creación transaccional de inscripción, roster, tarifa snapshot, pago, historial, PDF de solicitud y registro persistente de email.
- Carga, validación, revisión y descarga autorizada de comprobantes; administración de estado de pago e historial.
- Reportes agregados/detalle con control de rol y paginación en SQL.

• Rondas, enfrentamientos, puntajes auditables, combate actual, canales WebSocket y cronómetro configurable/persistente UP/DOWN.

• Reglamentos por evento/categoría, versionado, publicación y contenido cifrado en SQL con migraciones de backfill.

• Migrador con control de hashes de scripts y secuencia 01–32; pruebas automatizadas y colección/entorno de Postman.

Frontend

• Acceso del participante simplificado a Google y perfil con institución; el personal autorizado conserva acceso email/contraseña separado.

• Menús por perfil, Inicio del participante, etiquetas Ver combate/Combate en vivo, iconos y navegación responsive con menú móvil.

• Formulario de inscripción guiado por límites del evento/categoría, validación de integrantes, duplicidad y pasos de resumen/confirmación.

• Panel participante para inscripciones/órdenes, equipo, categorías y reglamento.

• Panel administrativo con vistas operativas, nombre de equipo visible, filtros, CSV, estados y vista previa del comprobante para revisión.

• Pantalla pública de combate consume lectura inicial + WebSocket; refleja marcador, estado y cronómetro del servidor.

Archivos principales a conocer: backend/uta_sumo_api/main.py, routers/auth.py, registrations.py, payments.py, reports.py, competition.py, regulations.py; database/scripts; frontend/uta-sumo-web/src/App.jsx, ModuloParticipante/ParticipantAccess.jsx, ModuloIncripcion/components/Inscripcion.jsx, ModuloAdministracion y ModuloResultados/Resultados.jsx.

14. Pendientes operativos y recomendaciones de despliegue

Tema	Antes de producción
Google OAuth	Autorizar origen de frontend exacto y dominio productivo; consent screen, usuarios de prueba/publicación, URL HTTPS. El Client ID frontend/backend debe coincidir.
SMTP	Definir remitente institucional, TLS, usuario/contraseña de aplicación; probar envío y reintento de outbox. No insertar password SMTP en Vite/frontend.
Secretos	Generar/guardar secretos en el gestor del host; quitar bootstrap demo; rotación controlada; no usar valores de ejemplo.
HTTPS y CORS	Certificado TLS, CORS limitado al dominio real, reverse proxy, cabeceras seguras y WebSocket upgrade del proxy.
SQL Server	Usuario mínimo necesario, backups cifrados probados, retención, índices revisados, monitoreo de conexiones y restauración.
Almacenamiento de pagos	Volumen privado con cuota, retención, backup, permisos OS y plan de eliminación conforme a política.
WebSocket escalado	Para más de un worker/instancia, incorporar broker compartido y monitorizar conexiones/reconexión.
Operación	Logs centralizados sin tokens/PII, alertas, límites de tasa, seguimiento de outbox y ensayo de migración en staging.
Privacidad	Definir base legal, consentimiento, retención/borrado, aviso de privacidad, contacto y proceso de incidentes.

Puntos de atención de documentación

Se detectó texto desactualizado en el README del frontend que aún describe registro manual del participante; la implementación visible actual usa Google como camino de registro/acceso del participante. También el README backend tiene un párrafo de roster anterior con 2 participantes y 0 coach: el último ajuste de migración 25 fija la regla oficial vigente en 2 participantes y 1 coach. La guía presente documenta el comportamiento actual del código y no reproduce esos dos textos antiguos.

Resumen para el equipo frontend

Usar los endpoints y contratos de esta guía como fuente de verdad. Para participante: Google Identity Services → ID token → POST /api/v1/auth/google; conservar el JWT recibido, mandar Authorization: Bearer en operaciones privadas; obtener reglas/tarifa desde API; consumir estados de pago, recibos, reglamento y resultados. Para el cronómetro/combate, conectar WebSocket público, aplicar instantánea inicial y reconectar si se cierra. Para reportes/admin, respetar separación de roles y controles de paginación; no mostrar datos financieros al rol operativo.

Referencias del checkout

Rama feature/backend-core · commit revisado 9521105 · guía contrastada con README backend, main.py, routers, migrador, scripts SQL 01–32 y componentes React vigentes. La guía no contiene contraseñas, SECRET_KEY, PII_ENCRYPTION_KEY ni SMTP_PASSWORD.